



ESCOLA DE INOVAÇÃO E COCRIAÇÃO
EM ENGENHARIA DE SOFTWARE
E CIÊNCIA DE DADOS

PROGRAMAÇÃO • RUBY

Ruby e Rails: do zero à primeira API REST

Uma aula de duas horas sobre a linguagem Ruby e os fundamentos do framework Rails, terminando com um endpoint REST funcionando.

Fundamentos de Ruby → Rails

2 horas

Agenda

01	Fundamentos de Ruby	35 min
02	Orientação a objetos	20 min
03	Gems e Bundler	10 min
04	Introdução ao Rails	15 min
05	Rotas, controllers e ActiveRecord	25 min
06	Construindo a API REST	15 min

Por que Ruby e Rails

- + Ruby é uma linguagem dinâmica e orientada a objetos, com sintaxe legível e foco em produtividade.
- + Rails é um framework web escrito em Ruby que segue "convenção sobre configuração": menos decisões, mais padrões.
- + A dupla é usada por empresas como GitHub, Shopify e Basecamp para construir aplicações web rapidamente.
- + Ao final desta aula você vai ter uma API REST simples rodando localmente.



01

BLOCO 01

Fundamentos de Ruby

Ambiente: irb e arquivos .rb

```
# no terminal, console interativo
$ irb
irb(main):001> puts "Olá, Ruby!"
Olá, Ruby!

# em um arquivo ola.rb
$ ruby ola.rb
```

irb é o console interativo do Ruby; ruby executa um arquivo .rb diretamente.

Variáveis e tipos de dados

```
nome = "Ana"           # String
idade = 28              # Integer
altura = 1.68           # Float
ativo = true            # TrueClass
lista = [1, 2, 3]       # Array
dados = { cidade: "Fortaleza" } # Hash
```

Ruby infere o tipo pelo valor atribuído — não é preciso declarar o tipo da variável.

Strings e interpolação

```
nome = "Ana"
saudacao = "Olá, #{nome}!"
puts saudacao           # Olá, Ana!

texto = "ruby"
puts texto.upcase       # RUBY
puts texto.length       # 4
puts "Ana" + " Silva"   # Ana Silva
```

`#{ }` insere o resultado de qualquer expressão Ruby dentro de uma string.

Arrays e Hashes

Arrays

```
frutas = ["maçã", "banana", "uva"]  
frutas.push("pera")  
frutas[0]      # "maçã"  
frutas.length  # 4
```

Hashes

```
pessoa = { nome: "Ana", idade: 28 }  
pessoa[:nome]      # "Ana"  
pessoa[:cidade] = "Fortaleza"
```


Métodos

```
def saudacao(nome)
  "Olá, #{nome}!"
end

puts saudacao("Ana")      # Olá, Ana!

def area_retangulo(base, altura)
  base * altura
end
```

O valor da última linha é retornado automaticamente — a palavra return é opcional.

Condicionais: if, unless, case

```
idade = 20

if idade >= 18
  puts "Maior de idade"
else
  puts "Menor de idade"
end

puts "Sem permissão" unless idade >= 18

case idade
when 0..12 then puts "Criança"
when 13..17 then puts "Adolescente"
else puts "Adulto"
end
```

Laços e iteradores: each, map, select

```
numeros = [1, 2, 3, 4, 5]

numeros.each { |n| puts n }

dobrados = numeros.map { |n| n * 2 }
# [2, 4, 6, 8, 10]

pares = numeros.select { |n| n.even? }
# [2, 4]
```

Blocos ({} ou do...end) são a forma mais comum de iterar em Ruby.

• AO VIVO

Vamos codar: calculadora simples

- 1 Criar um método `calcular(a, b, operacao)`
- 2 Usar `case` para somar, subtrair, multiplicar e dividir
- 3 Testar com `puts calcular(10, 2, "soma")`
- 4 Tratar o caso de divisão por zero



02

BLOCO 02

Orientação a Objetos

Classes e atributos

```
class Pessoa
  attr_accessor :nome, :idade
end

pessoa = Pessoa.new
pessoa.nome = "Ana"
pessoa.idade = 28
puts pessoa.nome      # Ana
```

`attr_accessor` gera automaticamente os métodos de leitura e escrita (getters e setters).

initialize e métodos de instância

```
class Pessoa
  attr_accessor :nome, :idade

  def initialize(nome, idade)
    @nome = nome
    @idade = idade
  end

  def apresentar
    "Sou #{@nome}, tenho #{@idade} anos."
  end
end

ana = Pessoa.new("Ana", 28)
puts ana.apresentar
```

initialize é o construtor, chamado por Pessoa.new. @nome é uma variável de instância.

Herança

```
class Animal
  def initialize(nome)
    @nome = nome
  end

  def emitir_som
    "..."
  end
end

class Cachorro < Animal
  def emitir_som
    "Au au!"
  end
end
```

< indica herança. Cachorro reaproveita initialize e sobrescreve emitir_som.

- AO VIVO

Vamos codar: classe Livro

- 1 Criar a classe `Livro` com `titulo`, `autor` e `paginas`
- 2 Adicionar um método `resumo` que monta uma frase
- 3 Criar dois livros e imprimir seus resumos
- 4 Adicionar um método `lido?` que retorna `true` ou `false`



03

BLOCO 03

Gems e Bundler

O que é uma gem

- + Uma gem é um pacote de código Ruby reutilizável — como uma biblioteca.
- + Existem gems para quase tudo: envio de e-mails, autenticação, geração de PDF, testes.
- + O próprio Rails é distribuído como uma gem.
- + Gems são instaladas com *gem install nome_da_gem* ou, em projetos, via Bundler.

Bundler e o Gemfile

```
# Gemfile
source "https://rubygems.org"

gem "rails", "~> 7.1"
gem "sqlite3"
gem "puma"

# instala as gems listadas no Gemfile
$ bundle install
```

Bundler resolve as versões compatíveis entre as gems e trava o resultado no Gemfile.lock.



04

BLOCO 04

Introdução ao Rails

Rails e "convention over configuration"

- + Rails segue convenções de nomes e pastas padronizadas, evitando decisões repetitivas.
- + Arquitetura MVC: Model (dados), View (apresentação), Controller (regras da requisição).
- + Um model Product gera, por convenção, a tabela products, o ProductsController e as views correspondentes.
- + Isso torna o Rails muito produtivo para criar APIs e aplicações web rapidamente.

Estrutura MVC de um projeto

```
app/  
  controllers/  
    items_controller.rb  
  models/  
    item.rb  
  views/  
config/  
  routes.rb  
  database.yml  
db/  
  migrate/
```

Requisição → routes.rb decide qual controller responde → controller conversa com o model → resposta é montada.

Criando o projeto: rails new

```
$ gem install rails  
$ rails new minha_api --api  
$ cd minha_api  
$ bin/rails server
```

—api gera um projeto mais leve, sem views HTML — ideal para uma API REST.



05

BLOCO 05

Rotas, Controllers e ActiveRecord

config/routes.rb

```
# config/routes.rb
Rails.application.routes.draw do
  resources :items, only: [:index, :show, :create]
end

# gera:
# GET    /items      items#index
# GET    /items/:id  items#show
# POST   /items      items#create
```

resources cria as rotas RESTful padrão de uma vez, seguindo a convenção Rails.

Controllers e actions

```
# app/controllers/items_controller.rb
class ItemsController < ApplicationController
  def index
    @items = Item.all
    render json: @items
  end

  def show
    @item = Item.find(params[:id])
    render json: @item
  end
end
```

Cada action responde a uma rota; render json: converte o objeto Ruby em JSON.

• AO VIVO

Vamos codar: primeira rota

- 1 Adicionar `get "hello", to: "welcome#index"` em `routes.rb`
- 2 Criar o `WelcomeController` com a action `index`
- 3 Responder `render json: { mensagem: "olá" }`
- 4 Testar em `localhost:3000/hello`

Models e migrations

```
$ bin/rails generate model Item name:string price:decimal
```

```
# db/migrate/..._create_items.rb
```

```
class CreateItems < ActiveRecord::Migration[7.1]
```

```
  def change
```

```
    create_table :items do |t|
```

```
      t.string :name
```

```
      t.decimal :price
```

```
      t.timestamps
```

```
    end
```

```
  end
```

```
end
```

```
$ bin/rails db:migrate
```

Uma migration descreve uma mudança no banco; db:migrate aplica essa mudança.

Consultas básicas: find, where, create

```
Item.all
Item.find(1)
Item.where(name: "Caneta")
Item.create(name: "Caderno", price: 12.5)

item = Item.find(1)
item.price = 15.0
item.save
```

ActiveRecord traduz esses métodos Ruby em SQL — sem escrever uma linha de SQL.

- AO VIVO

Vamos codar: model Item

- 1 Gerar o model Item com name e price
- 2 Rodar a migration com `bin/rails db:migrate`
- 3 Abrir `bin/rails console` e criar alguns itens
- 4 Consultar os itens criados com `Item.all`



06

BLOCO 06

Construindo a API REST

Endpoint GET /items

```
# config/routes.rb
resources :items, only: [:index]

# app/controllers/items_controller.rb
class ItemsController < ApplicationController
  def index
    render json: Item.all
  end
end
```

Essas poucas linhas já formam um endpoint REST completo: rota, controller e resposta em JSON.

Testando a API com curl

```
$ bin/rails server

$ curl http://localhost:3000/items

[
  { "id": 1, "name": "Caderno", "price": "12.5" },
  { "id": 2, "name": "Caneta", "price": "3.0" }
]
```

Com o servidor rodando, qualquer cliente HTTP — curl, Postman, o navegador — pode consumir a API.

ENCERRAMENTO

Você construiu sua primeira API REST em Rails

O que vimos

- + Ruby: variáveis, métodos, classes e objetos
- + Rails: MVC, rotas, controllers e ActiveRecord
- + Um endpoint GET /items funcionando e testado com curl

Para continuar

- + CRUD completo: create, update e destroy
- + Validações e testes automatizados (RSpec, Minitest)
- + Autenticação (Devise, JWT)
- + Deploy (Render, Fly.io, Heroku)